



MARMARA
UNIVERSITY

CSE 1145 Introduction to Computer Programming

Laboratory Manual

Res. Asst. Oğuzhan ÇİFTÇİ
oguzhan.ciftci at marmara.edu.tr

DEPARTMENT OF ELECTRICAL & ELECTRONICS
ENGINEERING

FALL 2025-2026

v2.1

Contents

Lab #1	2
Lab #2	6
Lab #3	12
Lab #4	20
Lab #5	27
Lab #6	33
Lab #7	44
Lab #8	50
Lab #9	58
Lab #10	65
Lab #11	71
Lab #12	79
Lab #13	88
Lab #14	96
REFERENCES	105

Lab #1

In this introductory lab session, you will learn the following fundamental concepts of C programming:

- **Program Structure:** Understanding the basic skeleton of a C program, including the use of `#include <stdio.h>` and the `main()` function.
- **Standard Output:** Using the `printf()` function to display text strings and formatted messages on the console.
- **Escape Sequences:** Learning how to use the newline character (`\n`) for proper output formatting.
- **Variable Declaration:** Defining integer variables (`int`) to store and manipulate data.
- **User Input:** Using the `scanf()` function to read numerical data provided by the user via the keyboard.
- **Basic Arithmetic Operations:** Performing simple mathematical calculations, such as addition (+), to process numerical values.
- **Conditional Logic:** Implementing the `if-else` structure to allow the program to make decisions based on specific conditions.
- **Modulus Operator:** Utilizing the `%` operator to calculate remainders, specifically for determining numerical properties like odd and even.

Question 1.1

Write a program that displays following sentences:

- Hello World!
- Welcome to CSE1145 Course!
- Programming is fun!

```
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("Hello World!\n");
6     printf("Welcome to CSE1145 Course!\n");
7     printf("Programming is fun!\n");
8     return 0;
9 }
```

Question 1.2

Write a complete C Program that reads two numbers from the user and add them, an then print the result on the screen.

For example :

Input	Result
3 5	8

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int a, b, c;
6
7     //printf("Enter two numbers: ");
8     scanf("%d%d", &a, &b);
9     c=a+b;
10    //printf("Result is %d ", c);
11    printf("%d", c);
12    return 0;
13 }
```

Question 1.3

Write a complete C Program where you read a number from the user and then decide whether it is odd or even.

For example :

Input	Result
13	13 is an odd number.
2456	2456 is an even number.

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int a;
6     //printf("Enter a number: ");
7     scanf("%d", &a);
8     if (a%2==0)
9         printf("%d is an even number.\n", a);
10    else
11        printf("%d is an odd number.\n", a);
12    return 0;
13 }
```

Lab #2

In this lab session, the focus is on data types, memory operations, and formatted input/output:

- **Floating-Point Arithmetic:** Working with `double` and `float` types for high-precision calculations.
- **Formatted Output:** Using precision specifiers (e.g., `%.21f`) to control the number of digits displayed after the decimal point.
- **Character Handling & ASCII:** Understanding how characters are stored as integer values and learning how to retrieve ASCII codes.
- **Integer Division:** Differentiating between the quotient (`/`) and the remainder (`%`) in mathematical operations.
- **Memory Measurement:** Utilizing the `sizeof` operator to determine the memory footprint (in bytes) of different data types.
- **Swapping Logic:** Learning the algorithmic steps to exchange values between two variables using a third temporary variable.

Question 2.1

In this example, the product of two floating-point numbers entered by the user is calculated and printed on the screen.

For example :

Input	Result
2.4 1.12	Enter two numbers: 2.400000 1.120000 Product = 2.69

```
1 #include <stdio.h>
2 int main() {
3     double a, b, product;
4     printf("Enter two numbers: ");
5     scanf("%lf %lf", &a, &b);
6     printf("%lf %lf\n", a, b);
7
8     // Calculating product
9     product = a * b;
10
11    // %.2lf displays number up to 2 decimal point
12    printf("Product = %.2lf", product);
13
14    return 0;
15 }
```


Question 2.2

In this example, you will learn how to find the ASCII value of a character.

For example :

Input	Result
A	Enter a character: ASCII value of A = 65

```
1 #include <stdio.h>
2 int main() {
3     char c;
4     printf("Enter a character: ");
5     scanf("%c", &c);
6     //printf("%c\n", c);
7
8     // %d displays the integer value of a character
9     // %c displays the actual character
10    printf("ASCII value of %c = %d", c, c);
11
12    return 0;
13 }
```

Question 2.3

In this example, you will learn to find the quotient and remainder when an integer is divided by another integer.

For example :

Input	Result
25 4	Enter dividend: 25 Enter divisor: 4 Quotient = 6 Remainder = 1

```
1 #include <stdio.h>
2 int main() {
3     int dividend, divisor, quotient, remainder;
4     printf("Enter dividend: ");
5     scanf("%d", &dividend);
6     printf("%d\n", dividend);
7     printf("Enter divisor: ");
8     scanf("%d", &divisor);
9     printf("%d\n", divisor);
10
11     // Computes quotient
12     quotient = dividend / divisor;
13
14     // Computes remainder
15     remainder = dividend % divisor;
16
17     printf("Quotient = %d\n", quotient);
18     printf("Remainder = %d", remainder);
19     return 0;
20 }
```

Question 2.4

In this example, you will learn to evaluate the size of each variable using sizeof operator.

```
1 #include<stdio.h>
2 int main() {
3     int intType;
4     float floatType;
5     double doubleType;
6     char charType;
7
8     // sizeof evaluates the size of a variable
9     printf("Size of int: %zu bytes\n", sizeof(intType));
10    printf("Size of float: %zu bytes\n", sizeof(floatType));
11    printf("Size of double: %zu bytes\n", sizeof(doubleType))
12    ;
13    printf("Size of char: %zu byte\n", sizeof(charType));
14
15    return 0;
16 }
```

Question 2.5

In this example, you will learn to swap two numbers in C programming.

For example :

Input	Result
1.2 2.45	Enter first number: 1.200000 Enter second number: 2.450000 After swapping, first number = 2.45 After swapping, second number = 1.20

```
1 #include<stdio.h>
2 int main() {
3     double first, second, temp;
4     printf("Enter first number: ");
5     scanf("%lf", &first);
6     printf("%lf\n", first);
7     printf("Enter second number: ");
8     scanf("%lf", &second);
9     printf("%lf\n", second);
10    // value of first is assigned to temp
11    temp = first;
12
13    // value of second is assigned to first
14    first = second;
15
16    // value of temp (initial value of first) is assigned to
17    // second
18    second = temp;
19
20    // %.2lf displays number up to 2 decimal points
21    printf("\nAfter swapping, first number = %.2lf\n", first);
22    printf("After swapping, second number = %.2lf", second);
23    return 0;
24 }
```

Lab #3

In this lab session, students will explore the various types of operators and their roles in forming expressions in C:

- **Increment and Decrement:** Mastering unary operators (`++` and `--`) to modify variable values by one.
- **Assignment Operations:** Understanding the standard assignment operator (`=`) and shorthand compound assignments (e.g., `+=`, `*=`, `%=`).
- **Relational Operators:** Comparing operands using `==`, `!=`, `>`, `<`, `>=`, and `<=` to evaluate truth values (0 or 1).
- **Logical Operators:** Using Boolean logic (`&&`, `||`, and `!`) to construct complex conditional expressions.
- **Character Arithmetic:** Applying ASCII distance concepts to perform character transformations, such as case conversion.
- **Mathematical Conditions:** Implementing divisibility checks using the modulus operator to solve basic numerical problems.

Question 3.1

C programming has two operators increment ++ and decrement – to change the value of an operand (constant or variable) by 1.

Increment ++ increases the value by 1 whereas decrement – decreases the value by 1. These two operators are unary operators, meaning they only operate on a single operand.

Take two integer and two float values. Then show the increment and decrement operators' effects as shown in below example.

For example :

Input	Result
10 100 10.5 100.5	a = 10, ++a = 11 b = 100, --b = 99 c = 10.500000, ++c = 11.500000 d = 100.500000, --d = 99.500000

```
1 #include <stdio.h>
2 int main()
3 {
4     int a ,b;
5     float c, d;
6
7     scanf("%d%d", &a,&b);
8     scanf("%f%f", &c,&d);
9
10    printf("a = %d, ", a);
11    printf("++a = %d \n", ++a);
12
13    printf("b = %d, ", b);
14    printf("--b = %d \n", --b);
15
16    printf("c = %f, ", c);
17    printf("++c = %f \n", ++c);
18
19    printf("d = %f, ", d);
20    printf("--d = %f \n", --d);
21
22
23    return 0;
24 }
```

Question 3.2

An assignment operator is used for assigning a value to a variable. The most common assignment operator is =

Operator	Example	Sometimes
=	a = b	a = b
+=	a += b	a = a+b
-=	a -= b	a = a-b
*=	a *= b	a = a*b
/=	a /= b	a = a/b
%=	a %= b	a = a%b

Take one integer value from the user. Then try all the operators above.

For example :

Input	Result
5	c = 5
	c = 10
	c = 5
	c = 25
	c = 5
	c = 0

```
1 #include <stdio.h>
2 int main()
3 {
4     int a, c;
5
6     scanf("%d", &a);
7
8     c = a;
9     printf("c = %d\n", c);
10    c += a;
11    printf("c = %d\n", c);
12    c -= a;
13    printf("c = %d\n", c);
14    c *= a;
15    printf("c = %d\n", c);
16    c /= a;
17    printf("c = %d\n", c);
18    c %= a;
19    printf("c = %d\n", c);
20
21    return 0;
22 }
```

Question 3.3

A relational operator checks the relationship between two operands. If the relation is true, it returns 1; if the relation is false, it returns value 0.

Operator	Meaning of Operator	Example
<code>==</code>	Equal to	<code>5 == 5</code> is evaluated to 1
<code>></code>	Greater than	<code>5 > 3</code> is evaluated to 1
<code><</code>	Less than	<code>5 < 3</code> is evaluated to 0
<code>!=</code>	Not equal to	<code>5 != 3</code> is evaluated to 1
<code>>=</code>	Greater than or equal to	<code>5 >= 3</code> is evaluated to 1
<code><=</code>	Less than or equal to	<code>5 <= 3</code> is evaluated to 0

Take three integer numbers and try all relational operators above.

For example :

Input	Result
5 5 10	5 == 5 is 1
	5 == 10 is 0
	5 > 5 is 0
	5 > 10 is 0
	5 < 5 is 0
	5 < 10 is 1
	5 != 5 is 0 ...

```
1 #include <stdio.h>
2 int main()
3 {
4     int a, b, c;
5     scanf("%d%d%d", &a, &b, &c);
6
7     printf("%d == %d is %d \n", a, b, a == b);
8     printf("%d == %d is %d \n", a, c, a == c);
9     printf("%d > %d is %d \n", a, b, a > b);
10    printf("%d > %d is %d \n", a, c, a > c);
11    printf("%d < %d is %d \n", a, b, a < b);
12    printf("%d < %d is %d \n", a, c, a < c);
13    printf("%d != %d is %d \n", a, b, a != b);
14    printf("%d != %d is %d \n", a, c, a != c);
15    printf("%d >= %d is %d \n", a, b, a >= b);
16    printf("%d >= %d is %d \n", a, c, a >= c);
17    printf("%d <= %d is %d \n", a, b, a <= b);
18    printf("%d <= %d is %d \n", a, c, a <= c);
19
20    return 0;
21 }
```


Question 3.4

An expression containing logical operator returns either 0 or 1 depending upon whether expression results true or false.

Operator	Meaning	Example
&&	Logical AND. True only if all operands are true	If c = 5 and d = 2 then, expression ((c==5) && (d!=2)) equals to 0.
	Logical OR. True only if either one operand is true	If c = 5 and d = 2 then, expression ((c==5) (d!=2)) equals to 1.
!	Logical NOT. True only if the operand is 0	If c = 5 then, expression !(c==5) equals to 0.

Consider the following example and write the code for it.

For example :

Input	Result
5 5 10	(a == b) && (c > b) is 1
	(a == b) && (c < b) is 0
	(a == b) (c < b) is 1
	(a != b) (c < b) is 0
	!(a != b) is 1
	!(a == b) is 0

```
1 #include <stdio.h>
2 int main()
3 {
4     int a, b, c, result;
5     scanf("%d%d%d", &a, &b, &c);
6     result = (a == b) && (c > b);
7     printf("(a == b) && (c > b) is %d \n", result);
8     result = (a == b) && (c < b);
9     printf("(a == b) && (c < b) is %d \n", result);
10    result = (a == b) || (c < b);
11    printf("(a == b) || (c < b) is %d \n", result);
12    result = (a != b) || (c < b);
13    printf("(a != b) || (c < b) is %d \n", result);
14    result = !(a != b);
15    printf("!(a != b) is %d \n", result);
16    result = !(a == b);
17    printf("!(a == b) is %d \n", result);
18    return 0;
19 }
```

Question 3.5

Write a program which reads a lower-case char (character) from the user, then prints its upper-case version on the screen.

Hint: Note that in the ASCII table, the distance between a lower-case letter and its corresponding upper-case version is the same for all letters.

For example :

Input	Result
a	A

```
1 #include <stdio.h>
2 int main(){
3     char input;
4     scanf("%c",&input);
5     printf("%c",input - ('a'-'A'));
6 }
```

Question 3.6

Write a program which prints the square of the given number, if the number is divisible by seven. Otherwise print the statement "not multiple of 7".

For example :

Input	Result
8	not multiple of 7
7	49

```
1 #include <stdio.h>
2
3 int main() {
4     int k;
5     scanf("%d",&k);
6     if (k%7 == 0)
7         printf("%d",k*k);
8     else
9         printf("not multiple of 7");
10 }
```

Question 3.7

Ask three float (or double) numbers to enter from the user.
Then find the largest number among these three numbers by using if statement.
Consider the following example.

For example :

Input	Result
5.25 125.2 1852.39	Enter three different numbers: 5.250000 125.200000 1852.390000 1852.39 is the largest number.

```
1 #include <stdio.h>
2
3 int main() {
4
5     double n1, n2, n3;
6
7     printf("Enter three different numbers: ");
8     scanf("%lf %lf %lf", &n1, &n2, &n3);
9
10    printf("%lf %lf %lf\n", n1, n2, n3);
11
12    // if n1 is greater than both n2 and n3, n1 is the largest
13    if (n1 >= n2 && n1 >= n3)
14        printf("%.2f is the largest number.", n1);
15
16    // if n2 is greater than both n1 and n3, n2 is the largest
17    if (n2 >= n1 && n2 >= n3)
18        printf("%.2f is the largest number.", n2);
19
20    // if n3 is greater than both n1 and n2, n3 is the largest
21    if (n3 >= n1 && n3 >= n2)
22        printf("%.2f is the largest number.", n3);
23
24    return 0;
25 }
```

Lab #4

In this lab session, students will focus on advanced control flow and multi-conditional decision-making structures:

- **Multi-conditional Logic:** Implementing `if-else if-else` ladders to handle programs with multiple potential outcomes.
- **Input Validation:** Learning to verify if user inputs fall within logical or physical constraints (e.g., age or grade ranges).
- **Nested Decision Structures:** Developing complex algorithms by nesting conditional statements to solve hierarchical problems, such as leap year calculations.
- **Selection with Switch-Case:** Utilizing the `switch` structure as an efficient alternative for equality-based decision-making.
- **Character Analysis:** Applying logical operators to classify characters into groups, such as vowels, consonants, or invalid entries.
- **Menu-Driven Programming:** Creating functional structures like simple calculators that respond to specific character-based operators.

Question 4.1

Write a C program to check whether a person is eligible to vote or not in Türkiye.

Hint: The minimum age for a person having right to vote and stand for election is 18 since 2017.

Hint: Take 1 integer input from the user as his/her age.

Hint: Entered age must be a positive number.

For example :

Input	Result
25	Enter your age:25 You are eligible to vote.
13	Enter your age:13 Sorry, you cannot vote.
-5	Enter your age:-5 Please enter a valid age.

```
1 #include <stdio.h>
2 int main()
3 {
4     int age;
5     printf("Enter your age:");
6     scanf("%d",&age);
7     printf("%d\n",age);
8
9     if(age>=18)
10    {
11        printf("You are eligible to vote.");
12    }
13    else if(age<0){
14        printf("Please enter a valid age.");
15    }
16    else
17    {
18        printf("Sorry, you cannot vote.");
19    }
20 }
```

Question 4.2

Write a C program to calculate the grade of the student according to the specified marks.

The grade must be an integer value between 0-100.

Hint: Take 1 integer grade from the user.

Hint: Use the following table to specify the letter grades.

AA	[80 – 100]
BA	[71 – 80]
BB	[63 – 71]
CB	[55 – 63]
CC	[50 – 55]
DC	[45 – 50]
DD	[35 – 45]
FF	[0 – 35]

For example :

Input	Result
77	Enter your mark: 77 Your grade is BA
54	Enter your mark: 54 Your grade is CC
101	Enter your mark: 101 The grade must be an integer value between 0-100

```
1 #include <stdio.h>
2 int main()
3 {
4     int marks;
5     printf("Enter your mark: ");
6     scanf("%d",&marks);
7     printf("%d\n",marks);
8
9     if(marks >= 0 && marks <= 100){
10
11         if(marks > 79 && marks <= 100)
12             printf("Your grade is AA");
13         else if (marks > 70 && marks <= 79)
14             printf("Your grade is BA");
15         else if (marks > 62 && marks <= 70)
16             printf("Your grade is BB");
17         else if (marks > 54 && marks <= 62)
18             printf("Your grade is CB");
19         else if (marks > 49 && marks <= 54)
```

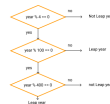
```
20     printf("Your grade is CC");
21 else if (marks > 44 && marks <= 49)
22     printf("Your grade is DC");
23 else if (marks > 34 && marks <= 44)
24     printf("Your grade is DD");
25 else
26     printf("Sorry, you have failed");
27 }
28 else
29     printf("The grade must be an integer value between
30 0-100");
31 }
```


Question 4.3

Complete the following C program that checks a year is a leap year or not.
Hint1: A leap year is a calendar year containing one additional day (Feb 29th) added to keep the calendar year synchronized with the astronomical year. That is, a leap year has 366 days instead of the usual 365 days that an ordinary year has.

Hint2: A leap year is a year whose number is perfectly divisible by four - except for years which are both divisible by 100 and not divisible by 400. The second part of the rule effects century years. For example; the century years 1600 and 2000 are leap years, but the century years 1700, 1800, and 1900 are not.

You can use the following algorithm:



For example :

Input	Result
1600	Leap
1800	Not Leap

```
1 #include <stdio.h>
2 int main()
3 {
4     int year;
5     scanf("%d",&year);
6     if(year%4 == 0){
7         if(year%100 == 0){
8             if ( year%400 == 0)
9                 printf("Leap");
10            else
11                printf("Not Leap");
12        }
13    }
14    else
15        printf("Leap");
16 }
17 else
18     printf("Not Leap");
19 return 0;
20 }
```

Question 4.4

Write a C program to check the entered letter is a vowel or a consonant.
Hint: Take 1 character input from the user and make sure that it is a letter indeed. Otherwise, give an error message of "Invalid entry!".

For example :

Input	Result
f	Enter an alphabet: f f is a consonant.
2	Enter an alphabet: 2 Invalid entry!

```
1 #include <stdio.h>
2 int main() {
3     char c;
4     int lowercase_vowel, uppercase_vowel;
5     printf("Enter an alphabet: ");
6     scanf("%c", &c);
7     printf("%c\n", c);
8     if( !( (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') ) ){
9         printf("Invalid entry!");
10    }
11    else{
12        // evaluates to 1 if variable c is a lowercase vowel
13        lowercase_vowel = (c == 'a' || c == 'e' || c == 'i' || c
14        == 'o' || c == 'u');
15        // evaluates to 1 if variable c is a uppercase vowel
16        uppercase_vowel = (c == 'A' || c == 'E' || c == 'I' || c
17        == 'O' || c == 'U');
18        // evaluates to 1 (true) if c is a vowel
19        if (lowercase_vowel || uppercase_vowel)
20            printf("%c is a vowel.", c);
21        else
22            printf("%c is a consonant.", c);
23    }
24    return 0;
25 }
```

Question 4.5

Write a simple calculator program that reads operators (+, -, *, /) and double operands from the user and makes the proper calculation. Note that use the operands with only one decimal digit in the operations.

Hint: Use switch-case structure.

For example :

Input	Result
+ 1.2 3.456	1.2 + 3.5 = 4.7
* 3.452 67.432	3.5 * 67.4 = 232.8

```
1 // Program to create a simple calculator
2 #include <stdio.h>
3 int main() {
4     char operator;
5     double n1, n2;
6     scanf("%c", &operator);
7     scanf("%lf %lf",&n1, &n2);
8     switch(operator)
9     {
10         case '+':
11             printf("%.1lf + %.1lf = %.1lf",n1, n2, n1+n2);
12             break;
13         case '-':
14             printf("%.1lf - %.1lf = %.1lf",n1, n2, n1-n2);
15             break;
16         case '*':
17             printf("%.1lf * %.1lf = %.1lf",n1, n2, n1*n2);
18             break;
19         case '/':
20             printf("%.1lf / %.1lf = %.1lf",n1, n2, n1/n2);
21             break;
22         default:
23             printf("Error! operator is not correct");
24     }
25     return 0;
26 }
```

Lab #5

In this lab session, students will explore advanced selection structures and the fundamental mechanics of iterative loops:

- **Advanced Selection:** Implementing `switch-case` within conditional statements to handle categorical data, such as classifying alphabetic characters.
- **Introduction to Loops:** Mastering the `for` loop structure to perform repeated calculations and automate sequential tasks.
- **Mathematical Algorithms:** Developing iterative logic for numerical operations, including factorial computations and power functions (a^b).
- **Sequence Implementation:** Constructing logic for generating series, such as the Fibonacci sequence, using loop-based state updates.
- **Range-Based Summation:** Utilizing loops to traverse specific numerical intervals and apply conditional filters, such as summing only odd numbers within a range.

Question 5.1

Write a simple C program that reads a character from user and check whether it is VOWEL or CONSONANT if entered character was an alphabet using switch case statement.

For example :

Input	Result
a	a is a VOWEL.
2	2 is not a LETTER.
z	z is a CONSONANT.

```
1  #include <stdio.h>
2  int main()
3  {
4      char ch;
5      scanf("%c",&ch);
6      //condition to check character is alphabet or not
7      if((ch>='A' && ch<='Z') || (ch>='a' && ch<='z'))
8      { //check for VOWEL or CONSONANT
9          switch(ch)
10         {
11             case 'A':
12             case 'E':
13             case 'I':
14             case 'O':
15             case 'U':
16             case 'a':
17             case 'e':
18             case 'i':
19             case 'o':
20             case 'u':
21                 printf("%c is a VOWEL.\n",ch);
22                 break;
23             default:
24                 printf("%c is a CONSONANT.\n",ch);
25         }
26     }
27     else
28     {
29         printf("%c is not a LETTER.\n",ch);
30     }
31     return 0;
32 }
```

Question 5.2

Write a complete C program that prints the value of factorial according to given input.

For example :

Input	Result
5	120

```
1 #include <stdio.h>
2 int main()
3 {
4     int a, b, result=1, i;
5     scanf("%d %d", &a, &b);
6     for (i=0; i<b; i++){
7         result *= a;
8     }
9     printf("%d",result);
10    return 0;
11 }
```

Question 5.3

Write a complete C program which finds the a to the power of b using for loop and print the result on the screen.

For example :

Input	Result
5 2	25

```
1 #include <stdio.h>
2 int main()
3 {
4     int a, b, result=1, i;
5     scanf("%d %d", &a, &b);
6     for (i=0; i<b; i++){
7         result *= a;
8     }
9     printf("%d",result);
10    return 0;
11 }
```

Question 5.4

Fibonacci numbers are: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34,

(Each number is sum of the previous two numbers except the first two numbers 0 and 1 which are predefined)

Write a complete C program which reads a positive integer number N (smaller than 50), and prints the N.th fibonacci number.

For example :

Input	Result
5	First 5 terms of Fibonacci series are : 0 1 1 2 3
7	First 7 terms of Fibonacci series are : ... 5 8

```
1 #include<stdio.h>
2 int main()
3 {
4     int n, first = 0, second = 1, next, c;
5     scanf("%d",&n);
6     printf("First %d terms of Fibonacci series are :\n",n);
7     for ( c = 0 ; c < n ; c++ )
8     {
9         if ( c <= 1 )
10             next = c;
11         else
12         {
13             next = first + second;
14             first = second;
15             second = next;
16         }
17         printf("%d\n",next);
18     }
19     return 0;
20 }
```


Question 5.5

Develop a complete C program to find sum of odd numbers for a given range (from m to n).

For example :

Input	Result
5 7	Sum of odd numbers = 12

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 int main()
4 {
5     int i, m, n, sum=0;
6     scanf("%d", &m);
7     scanf("%d", &n);
8     for(i=(m % 2 == 0 ? m+1 : m); i<=n; i+=2)
9     {
10         sum += i;
11     }
12     printf("Sum of odd numbers = %d", sum);
13     return 0;
14 }
```

Lab #6

In this lab session, students will delve into user-defined data types, nested iterative structures, and conditional loop mechanics:

- **Enumerations (enum):** Defining custom data types to represent categorical values and integrating them with selection structures.
- **Nested Iterative Patterns:** Implementing loops within loops to generate complex geometric shapes and numerical patterns on the console.
- **Boundary Detection:** Applying conditional logic inside nested loops to distinguish between internal elements and boundaries of shapes.
- **While Loop Control:** Understanding the execution flow of `while` loops and the critical importance of update statements to prevent infinite execution.
- **Compound Loop Conditions:** Constructing loop headers with multiple logical operators to handle sophisticated termination criteria.
- **Algorithmic Number Processing:** Utilizing iterative structures to manipulate numerical data, specifically for tasks such as reversing integer values through digit extraction.

Question 6.1

Write a program that defines a type for letter grades (AA,BB,CC,DD,F as 4,3,2,1,0), reads a grade and displays corresponding letter grade. Use switch case; you are not allowed to use if-else statement for this question.

For example :

Input	Result
1	DD
3	BB
4	AA

```
1 #include <stdio.h>
2 enum grade_type {F,DD,CC,BB,AA};
3 int main()
4 {
5     enum grade_type g;
6     scanf("%u", &g);
7     switch (g)
8     {
9         case 4: printf("AA \n"); break;
10        case 3: printf("BB \n"); break;
11        case 2: printf("CC \n"); break;
12        case 1: printf("DD \n"); break;
13        case 0: printf("F \n"); break;
14        default: printf("Invalid \n"); break;
15    }
16    return 0;
17 }
```

Question 6.2

Fibonacci numbers are: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34,

(Each number is sum of the previous two numbers except the first two numbers 0 and 1 which are predefined)

Write a complete C program which reads a positive integer number N (smaller than 50), and prints the N.th fibonacci number.

For example :

Input	Result
5	First 5 terms of Fibonacci series are : 0 1 1 2 3

```
1 #include <stdio.h>
2 int main()
3 {
4
5     int n, first = 0, second = 1, next, c;
6     scanf("%d",&n);
7     printf("First %d terms of Fibonacci series are :\n",n);
8
9     for ( c = 0 ; c < n ; c++ )
10    {
11        if ( c <= 1 )
12            next = c;
13        else
14        {
15            next = first + second;
16            first = second;
17            second = next;
18        }
19        printf("%d\n",next);
20    }
21
22    return 0;
23 }
```

Question 6.3

Write a program which reads an integer, N, between 0 and 9. Then prints the numbers in the pattern as given in the following:

For example :

Input	Result
5	5 5 5 5 5 4 4 4 4 3 3 3 2 2 1

```
1 #include <stdio.h>
2 int main() {
3     int i, j, rows;
4     //int count = 1;
5     scanf("%d", &rows);
6     for (i = rows; i >= 0; i--) {
7         for (j = 1; j <= i; j++) {
8             printf("%d ", i);
9         }
10        printf("\n");
11    }
12    return(0);
13 }
```

Question 6.4

Write a program which reads two integers M, N then prints an MxN rectangle with zeros inside, but ones on the edges.

For example :

Input	Result
3 3	111 101 111

```
1 #include<stdio.h>
2 int main(){
3     int rows, cols , i, j;
4     scanf("%d %d", &rows, &cols);
5     /* Row iterator for loop */
6     for(i = 0; i < rows; i++){
7         /* Column iterator for loop */
8         for(j = 0; j < cols; j++){
9             /* Check if currently position is a boundary
10            position */
11             if(i==0 || i==rows-1 || j==0 || j==cols-1)
12                 printf("1");
13             else
14                 printf("0");
15         }
16         printf("\n");
17     }
18     return 0;
19 }
```

Question 6.5

Write a program in C to display the pattern like right angle triangle using an asterisk.

For example :

Input	Result
5	Input number of rows : * ** *** **** *****
11	Input number of rows : * ** *** *****

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int i,j,rows;
6     printf("Input number of rows : \n");
7     scanf("%d",&rows);
8     for(i=1;i<=rows;i++)
9     {
10        for(j=1;j<=i;j++)
11            printf("*");
12        printf("\n");
13    }
14 }
```

Question 6.6

Develop a complete C program to print star pattern for given row and column values.

You read row and column values from the user.

For example :

Input	Result
2 4	**** ****

```
1 #include <stdio.h>
2 int main()
3 {
4     int i, j, row, col;
5     /* Input number of rows from user */
6     scanf("%d", &row);
7     scanf("%d", &col);
8     /* Iterate through row rows */
9     for(i=1; i<=row; i++)
10    {
11        /* Iterate over col columns */
12        for(j=1; j<=col; j++)
13        {
14            /* Print star for each column */
15            printf("*");
16        }
17        /* Move to the next line/row */
18        printf("\n");
19    }
20    return 0;
21 }
```


Question 6.7

Develop a complete C program to print star pattern for given row and column values.

You read row and column values from the user.

For example :

Input	Result
10	value of a: 10 value of a: 11 value of a: 12 value of a: 13 value of a: 14 value of a: 15 value of a: 16 value of a: 17 value of a: 18 value of a: 19

```
1  #include <stdio.h>
2
3  int main () {
4
5      /* local variable definition */
6      int a ;
7
8      scanf("%d", &a);
9
10     /* while loop execution */
11     while( a < 20 ) {
12         printf("value of a: %d\n", a);
13         a++;
14     }
15
16     return 0;
17 }
```

Question 6.8

The fundamentals of while loops: be careful about infinite loops
Take a value from the user and print all the integer values from that value up until 5.
However, you must avoid infinite loops. Therefore, do not forget to increase the counter inside the loop.

For example :

Input	Result
4	4
	5

```
1 #include <stdio.h>
2 int main()
3 {
4     int var;
5     scanf("%d", &var);
6
7     while (var <=5)
8     {
9         printf("%d\n", var);
10        var++;
11    }
12    return 0;
13 }
```

Question 6.9

The fundamentals of while loops: usage of logical operators in while loops
Take 2 values from user. Print as long as the first taken value is less than or equal to 4 or the second taken value is less than or equal to 3.
Please check the code.

For example :

Input	Result
1 1	1 1
	2 2
	3 3
	4 4

```
1 #include <stdio.h>
2 int main()
3 {
4     int i, j;
5
6     scanf("%d %d", &i, &j);
7
8     while (i <= 4 || j <= 3)
9     {
10    printf("%d %d\n", i, j);
11    i++;
12    j++;
13    }
14    return 0;
15 }
```

Question 6.10

Write a program that reverse the entered number.

For example :

Input	Result
85	58
315	513

```
1 #include <stdio.h>
2 int main(){
3     int n, reversedNumber = 0, remainder;
4     scanf("%d", &n);
5     while(n != 0){
6         remainder = n%10;
7         reversedNumber = reversedNumber*10 + remainder;
8         n /= 10;
9     }
10    printf("Reversed Number = %d", reversedNumber);
11    return 0;
12 }
```

Lab #7

In this lab session, students will master the **do-while** iterative structure and apply complex logical reasoning to mathematical problem-solving:

- **Guaranteed Execution Loops:** Understanding the mechanics of the **do-while** loop to ensure at least one execution of the code block regardless of the initial condition.
- **Advanced Logical Filtering:** Constructing compound loop termination criteria using logical operators to manage multiple variables simultaneously.
- **Prime Number Discovery:** Developing algorithms to identify prime numbers within a given range through nested iteration and state monitoring variables (flags).
- **Dynamic Data Accumulation:** Implementing logic to continuously read and aggregate user data until a specific terminal value is encountered.
- **Pre-processor Directives:** Utilizing **#define** for defining mathematical constants, thereby improving code modularity and precision.
- **Formula Integration:** Combining user-driven character selection with geometric formulas to calculate properties like area, perimeter, and volume of different shapes.

Question 7.1

Write a program that reads a character ('a', 'p', or 'v') and radius R. It displays the area or perimeter of a circle with radius R, or the volume of a sphere.

For example :

Input	Result
a 2	Area of circle = 12.560000
p 3	Perimeter of circle = 18.840000

```
1 #include <stdio.h>
2 #define PI 3.14
3 char ch;    float R;
4
5 int main()
6 {
7     scanf("%c%f",&ch,&R);
8     switch(ch)
9     {
10         case 'a': printf("Area of circle = %f\n", PI*R*R);
11                     break;
12         case 'p': printf("Perimeter of circle = %f\n", 2*PI*R);
13                     break;
14         case 'v': printf("Volume of sphere = %f\n", (4/3)*(PI*R
15                         *R*R));
16                     break;
17         default:  printf("Invalid input\n");
18                     break;
19     }
20     return 0;
21 }
```

Question 7.2

Write a complete C program to find the prime numbers from 2 to N where N is a positive integer number and taken from the user.

For example :

Input	Result
100	2 is prime 3 is prime 5 is prime 7 is prime 97 is prime
-350	Invalid input.

```
1  #include <stdio.h>
2  int main()
3  {
4      int N, flag_var, primeCand, divisor;
5
6      //Store the range in variables using scanf
7      scanf("%d", &N);
8      if(N > 0){
9          for(primeCand=2; primeCand<=N; ++primeCand)
10         {
11             flag_var=0;
12             for(divisor=2; divisor<=primeCand/2; ++divisor)
13             {
14                 if(primeCand%divisor==0)
15                 {
16                     flag_var=1;
17                     break;
18                 }
19             }
20             if(flag_var==0)
21                 printf("%d is prime\n",primeCand);
22         }
23     }
24     else
25         printf("Invalid input.");
26
27     return 0;
28 }
```

Question 7.3

The fundamentals of while loops: usage of logical operators in while loops
Take 2 values from user. Print as long as the first taken value is less than or equal to 4 or the second taken value is less than or equal to 3.
Please check the code.

For example :

Input	Result
1 1	1 1
	2 2
	3 3
	4 4

```
1 #include <stdio.h>
2 int main()
3 {
4     int i, j;
5
6     scanf("%d %d", &i, &j);
7
8     while (i <= 4 || j <= 3)
9     {
10    printf("%d %d\n", i, j);
11    i++;
12    j++;
13    }
14    return 0;
15 }
```


Question 7.4

Write a complete C program to add numbers until the user enters zero.
Hint: You can use do-while loop .

For example :

Input	Result
1.5 2.4 -3.4 4.2 0	Enter a number: 1.50 Enter a number: 2.40 Enter a number: -3.40 Enter a number: 4.20 Enter a number: 0.00 Sum = 4.70

```
1 #include <stdio.h>
2 int main() {
3     double number, sum = 0;
4
5     // the body of the loop is executed at least once
6     do {
7         printf("Enter a number: ");
8         scanf("%lf", &number);
9         printf("%.2lf\n", number);
10        sum += number;
11    }
12    while(number != 0);
13
14    printf("Sum = %.2lf", sum);
15
16    return 0;
17 }
```

Question 7.5

Write a complete C program to add numbers until the user enters zero.
Hint: You can use do-while loop .

For example :

Input	Result
79	Enter a Number : 79 0 1 1 2 3 5 8 13 21 34 55
-1 -2 3	Enter a Number : -1 Enter a Number : -2 Enter a Number : 3 0 1 1 2 3

```
1 #include <stdio.h>
2 int main ()
3 {
4     int i, n, j, k;
5     do{
6         printf ("Enter a Number : ");
7         scanf ("%d", &n);
8         printf("%d\n", n);
9     }
10    while(n < 0);
11    i = 0;
12    j = 1;
13    printf ("%d %d", i, j);
14    k = i + j;
15    while (k <= n)
16    {
17        printf (" %d", k);
18        i = j;
19        j = k;
20        k = i + j;
21    }
22    return 0;
23 }
```

Lab #8

In this lab session, students will transition towards modular programming and low-level data manipulation:

- **Modular Programming with Functions:** Learning to define, declare, and call functions to break complex problems into manageable, reusable components.
- **Parameter Passing and Return Values:** Understanding how to send data to functions and retrieve calculated results using the `return` statement.
- **Bitwise Logic:** Utilizing bitwise operators (e.g., `&`) as efficient alternatives to arithmetic operations for hardware-level data checks.
- **Numerical System Conversion:** Implementing algorithms within functions to perform base conversions, specifically from decimal to binary representations.
- **State Evaluation Functions:** Developing specialized functions to verify mathematical properties of numbers, such as identifying Prime, Armstrong, or Perfect numbers.
- **Code Reusability:** Practicing the organization of code where repetitive tasks are abstracted into functional templates to improve readability and maintainability.

Question 8.1

Complete the following C program that finds the square of any number using the function.

For example :

Input	Result
10	The square of 10.000 is : 100.00

```
1 #include <stdio.h>
2
3 double square(double num)
4 {
5     return (num * num);
6 }
7 int main()
8 {
9     double n, num;
10    scanf("%lf", &num);
11    n = square(num);
12    printf("The square of %.3lf is : %.2f\n", num, n);
13    return 0;
14 }
```

Question 8.2

Complete the following C program that checks a given number is even or odd using the function. You are not allowed to use division or modulus operations! Hint: Instead, you can use bitwise operations. Note that the least significant bit of an odd number is always 1, whereas it is 0 for even numbers.

For example :

Input	Result
1234	even

```
1  #include <stdio.h>
2
3  //if the least significant bit is 1 the number is odd and 0
   the number is even
4  int checkOddEven(int n1)
5  {
6      return (n1 & 1); //The & operator does a bitwise and,
7  }
8
9  int main()
10 {
11     int n1;
12     scanf("%d", &n1);
13
14     // If checkOddEven() function returns 1 then the number
   is odd
15     if(checkOddEven(n1))
16     {
17         printf("odd\n");
18     }
19     else
20     {
21         printf("even\n");
22     }
23     return 0;
24 }
```

Question 8.3

Complete the following C program that finds the sum of the series such as $1!/1+2!/2+3!/3+4!/4+5!/5$ using functions.

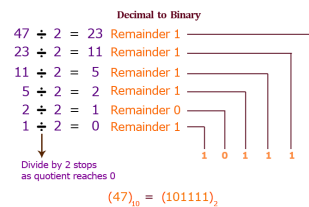
For example :

Input	Result
5	The sum of the series for 5 is : 34

```
1 #include <stdio.h>
2
3 int fact(int);
4 int main()
5 {
6     int i, sum=0, n;
7     scanf("%d", &n);
8     for(i=1; i<=n; i++){
9         sum+=fact(i)/i;
10    }
11    printf("The sum of the series for %d is : %d\n",n,sum);
12    return 0;
13 }
14 int fact(int n)
15 {
16     int num=0, f=1;
17     while(num<=n-1)
18     {
19         f =f+f*num;
20         num++;
21     }
22     return f;
23 }
```

Question 8.4

Complete the following C program that converts decimal number to binary number using the function. You can use the following algorithm:



For example :

Input	Result
47	101111

```
1 #include <stdio.h>
2
3 long toBin(int);
4
5 int main()
6 {
7     long bno;
8     int dno;
9     scanf("%d",&dno);
10    bno = toBin(dno);
11    printf("%ld",bno);
12    return 0;
13 }
14 long toBin(int dno)
15 {
16     long bno=0,remainder,f=1;
17     while(dno != 0)
18     {
19         remainder = dno % 2;
20         bno = bno + remainder * f;
21         f = f * 10;
22         dno = dno / 2;
23     }
24     return bno;
25 }
```

Question 8.5

Complete the following C program that checks whether a number is a prime number or not using the function.

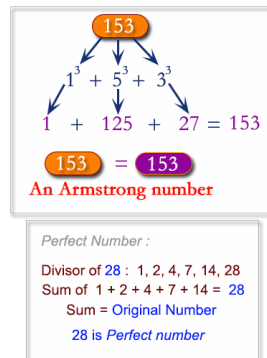
For example :

Input	Result
113	The number 113 is a prime number.

```
1 #include <stdio.h>
2 int PrimeOrNot(int);
3 int main()
4 {
5     int n1, prime;
6     scanf("%d", &n1);
7     prime = PrimeOrNot(n1);
8     if (prime == 1)
9         printf("The number %d is a prime number.\n", n1);
10    else
11        printf("The number %d is not a prime number.\n", n1);
12    return 0;
13 }
14 int PrimeOrNot(int n1)
15 {
16     int i = 2;
17     while (i <= n1 / 2)
18     {
19         if (n1 % i == 0)
20             return 0;
21         else
22             i++;
23     }
24     return 1;
25 }
```


Question 8.6

Complete the following C program that checks armstrong and perfect numbers using the function.



For example :

Input	Result
371	The 371 is an Armstrong number. The 371 is not a Perfect number.

```
1 #include <stdio.h>
2
3 int checkArmstrong(int n1);
4 int checkPerfect(int n1);
5
6 int main()
7 {
8     int n1;
9     scanf("%d", &n1);
10
11
12     //Calls the isArmstrong() function
13     if(checkArmstrong(n1))
14     {
15         printf("The %d is an Armstrong number.\n", n1);
16     }
17     else
18     {
19         printf("The %d is not an Armstrong number.\n", n1);
20     }
21
22     //Calls the checkPerfect() function
23     if(checkPerfect(n1))
24     {
```

```

25     printf("The %d is a Perfect number.\n\n", n1);
26 }
27 else
28 {
29     printf("The %d is not a Perfect number.\n\n", n1);
30 }
31 return 0;
32 }
33
34 // Checks whether a three digits number is Armstrong number
35 // or not.
36 // An Armstrong number is an n-digit number that is equal
37 // to the sum of the n-th powers of its digits.
38 int checkArmstrong(int n1)
39 {
40     int ld, sum, num;
41     sum = 0;
42     num = n1;
43     while(num!=0)
44     {
45         ld = num % 10; // find the last digit of the number
46         sum += ld * ld * ld; // calculate the cube of the
47         last digit and adds to sum
48         num = num/10;
49     }
50     return (n1 == sum);
51 }
52 // Checks whether the number is perfect number or not.
53 // a perfect number is a positive integer that is equal to
54 // the sum of its positive divisors excluding the number
55 // itself
56 int checkPerfect(int n1)
57 {
58     int i, sum, num;
59     sum = 0;
60     num = n1;
61     for(i=1; i<num; i++)
62     {
63         /* If i is a divisor of n1 */
64         if(num%i == 0)
65         {
66             sum += i;
67         }
68     }
69     return (n1 == sum);
70 }

```

Lab #9

In this lab session, students will explore variable visibility, pre-processor directives, and the foundational concepts of contiguous data storage using arrays:

- **Variable Scope and Lifetime:** Distinguishing between global and local variable declarations and understanding their accessibility across different program segments.
- **Function Execution Flow:** Mastering the interaction between the `main()` function and user-defined subroutines for structured program output.
- **Functional Power Calculations:** Implementing mathematical logic within functions to compute exponential values through iterative processes.
- **Macro Definitions:** Utilizing the `#define` directive to create parameterized macros for efficient, compile-time code substitutions.
- **Array Fundamentals:** Defining and allocating memory for single-dimensional arrays to store multiple elements of the same data type.
- **Array Manipulation:** Learning the techniques for populating arrays via user input and traversing them using loops for data retrieval and display.
- **Static Initialization:** Practicing the declaration of arrays with pre-defined values using brace-enclosed initializer lists.

Question 9.1

Write a complete C program that takes 2 inputs and prints sum of them.
(Initialize at least one variable as a global)

For example :

Input	Result
0 5	5

```
1 #include <stdio.h>
2 /* global variable declaration */
3 int g;
4 int main () {
5     /* local variable declaration */
6     int a, b;
7     /* actual initialization */
8     scanf("%d%d",&a,&b);
9     g = a + b;
10    printf ("%d",g);
11    return 0;
12 }
```

Question 9.2

Write a complete C program that prints text of "Main function." by using main function. Then writes "Welcome to my function. Feel at home." to the newline by using function.

For example :

Result
Main function. Welcome to my function. Feel at home.

```
1 #include <stdio.h>
2 void my_function();
3 int main(){
4     printf("Main function.\n");
5     my_function();
6     return 0; }
7 void my_function() {
8     printf("Welcome to my function. Feel at home.");
9 }
```

Question 9.3

Write a C program that gives the POWER of the numbers you entered.

For example :

Input	Result
5 2	25
0 62	0
-9 2	81

```
1 #include <stdio.h>
2 int power(int base, int exponent);
3 int main(){
4     int a,b,result;
5     scanf("%d",&a);
6     scanf("%d",&b);
7     result=power(a,b);
8     printf("%d",result);
9 }
10 int power(int base, int exponent) {
11     int result=1,i;
12     for(i=0;i<exponent;i++)
13         result*=base;
14     return result;
15 }
```

Question 9.4

Write a C program that returns the cube of a number with MACRO.

For example :

Input	Result
3	27
0	0
-4	-64

```
1 #include <stdio.h>
2 #define cube(x) (x*x*x)
3 int main(){
4     int in;
5     int result;
6     scanf("%d", &in);
7     result = cube(in);
8     printf("%d", result);
9     return 0;
10 }
```

Question 9.5

Fill an array with a loop according to entered numbers from user. You can define size of your array as 5.

For example :

Input	Result
1 2 3 4 5	1 2 3 4 5
-9 8 7 5 6	-9 8 7 5 6

```
1 #include <stdio.h>
2 int main () {
3     int n[5];
4     int i;
5     for (i=0;i<5;i++){
6         scanf("%d",&n[i]);
7     }
8     for (i=0;i<5;i++){
9         printf("%d ",n[i]);
10    }
11    return 0;
12 }
```


Question 9.6

Initialize an array during declaration, print all elements with loop and print it. Your array indexes are as shown in example below.

For example :

Result
19
10
8
17
9

```
1 #include <stdio.h>
2 int main(){
3     int test_array[5] = {19, 10, 8, 17, 9};
4     int i;
5     for(i=0;i<5;i++){
6         printf("%d\n",test_array[i]);
7     }
8 }
```

Lab #10

In this lab session, students will advance their skills in array manipulation, algorithmic data processing, and the representation of textual data:

- **Data Analysis in Arrays:** Implementing algorithms to identify extreme values (maximum) and calculate statistical measures like the average of array elements.
- **Dynamic Loop Boundaries:** Learning to handle variable-length data by reading the number of elements from the user before executing processing loops.
- **Functional Mapping:** Populating arrays based on mathematical formulas using loop indices to map values such as powers of two (2^i) to specific indices.
- **Array Reversal Techniques:** Understanding the logic required to flip the order of elements within an array using indexing and temporary storage variables.
- **String Representation:** Exploring character arrays as strings, including proper initialization and the critical role of the null terminator (`'\0'`).
- **Arrays in Functions:** Passing entire arrays as arguments to functions to perform modular tasks like printing or data transformation.

Question 10.1

Write a complete C program that finds the largest number of an array.

For example :

Input	Result
2 1 0 -1 8	8
87 4 5 698 2	698

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int i;
6     int arr[5];
7
8     for(i = 0; i < 5; ++i){
9         scanf("%d", &arr[i]);
10    }
11
12
13    for(i = 1; i <5; ++i)
14    {
15        if(arr[0] < arr[i])
16            arr[0] = arr[i];
17    }
18    printf("%d", arr[0]);
19
20    return 0;
21 }
```

Question 10.2

Write a complete C program that finds the average value of the elements in an array.

To do this, read the number of elements at first, then read the elements one by one.

For example :

Input	Result
3 2.1 3.1 4.4	3.20
5 41.8 54.7 2.0 5.8 96.3	40.12

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int n, i;
6     float num[100], sum = 0.0, average;
7     scanf("%d", &n);
8
9     for(i = 0; i < n; ++i)
10    {
11        scanf("%f", &num[i]);
12        sum += num[i];
13    }
14
15    average = sum / n;
16    printf("%.2f", average);
17
18    return 0;
19 }
```

Question 10.3

Write a C program Initialize a char array and print it using %s. Initialize a group of chars, you can find an example below.

char name[10] = 'L','e','s','s','o','n','s','\0'; // valid initialization

Your char array should include letters 'a', 'b', 'c' and 'd'.

```
1 #include <stdio.h>
2 int main () {
3
4     char c[5] = {'a', 'b', 'c', 'd', '\0'};
5     printf("our string is %s", c);
6 }
```

Question 10.4

Complete the following C program in order to fill the array with 2^i values. The value at index i will be 2^i . Read number of elements from the user, at first.

For example :

Input	Result
3	1 2 4

```
1 #include <stdio.h>
2 void printArray(int a[], int size);
3 int main () {
4     int i, n[100];
5     int numOfElements;
6     scanf("%d", &numOfElements);
7     n[0]=1;
8     for(i=1; i<numOfElements; i++ )
9         n[i] = n[i-1]*2;
10    printArray(n, numOfElements);
11    return 0;
12 }
13 void printArray(int a[], int size){
14     int j;
15     for (j=0; j<size; j++ )
16         printf("%d ", a[j]);
17 }
```

Question 10.5

Complete the following C program in order to fill the array with 2^i values. This time, the value at index $n-i$ will be 2^i . Read number of elements from the user, at first. (This will print the reverse of the list, i.e. the numbers in the list in decreasing order)

For example :

Input	Result
3	4 2 1

```
1 #include <stdio.h>
2 #define MAX_LENGTH 100
3 void printArray(int a[], int size);
4 void ReverseArray(int array[], int size);
5 int main () {
6     int i, n[MAX_LENGTH];
7     int numOfElements;
8     scanf("%d", &numOfElements);
9     n[0]=1;
10    for (i=1; i<numOfElements; i++ )
11        n[i] = n[i-1]*2;
12    ReverseArray(n, numOfElements);
13    printArray(n, numOfElements);
14    return 0;
15 }
16 void printArray(int a[], int size){
17     int j;
18     for (j=0; j<size; j++ )
19         printf("%d ", a[j]);
20 }
21 void ReverseArray(int array[], int size){
22     int i;
23     int temp;
24     for (i=0; i<size/2; i++) {
25         temp=array[i];
26         array[i]=array[size-i-1];
27         array[size-i-1] = temp;
28     }
29 }
```

Lab #11

In this lab session, students will explore advanced string manipulations, search strategies, and formal sorting algorithms:

- **Algorithmic Searching:** Developing logic to find specific combinations of elements within an array, such as identifying triplets that satisfy a target sum.
- **Mathematical Data Recovery:** Utilizing summation formulas to detect missing values within a defined numerical sequence.
- **Standard String Library (`string.h`):** Mastering core library functions including `strcpy` for copying, `strcat` for concatenation, and `strlen` for determining string length.
- **Manual Substring Processing:** Learning to isolate specific portions of a string by directly manipulating indices and ensuring proper null termination without library assistance.
- **Character Frequency Analysis:** Implementing traversal logic to count occurrences of specific characters within textual data.
- **Lexicographical Sorting:** Combining string comparison functions (`strcmp`) with the Bubble Sort algorithm to organize word lists in alphabetical (dictionary) order.

Question 11.1

This problem was asked by Microsoft.

Given an array of numbers and a number num, determine if there are three entries in the array which add up to the specified number num. For example, given [20, 303, 3, 4, 25] and num = 49, return true as $20 + 4 + 25 = 49$. To do this read num from the user at first.

For example :

Input	Result
49	$20 + 4 + 25 = 49$

```
1 #include <stdio.h>
2
3 int main() {
4     int i, j, k, num;
5     scanf("%d", &num);
6
7     int arr[] = {20, 303, 3, 4, 25};
8     int size = sizeof(arr)/sizeof(int);
9     for(i=0; i<size; i++){
10         for(j=0; j<size; j++){
11             for(k=0; k<size; k++){
12                 if (num==(arr[i] + arr[j] +arr[k])){
13                     printf("%d + %d + %d = %d", arr[i], arr[j],
14                         arr[k], num);
15                     return 0;
16                 }
17             }
18         }
19     }
20     return 0;
21 }
```

Question 11.2

Given an integer array with the size of 99 which contains numbers from 1 to 100 but one number is missing. Find that number.

You are given the array in the input, read them one by one using scanf inside a for loop.

For example :

Input	Result
85 41 ...	88

```
1 #include <stdio.h>
2
3 int main() {
4     int i, arr[99];
5     int sum=0;
6     int expectedSum = 100*101/2;
7
8     for(i=0; i<99; i++){
9         scanf("%d", &arr[i]);
10    }
11
12    for(i=0; i<99; i++){
13        sum+=arr[i];
14    }
15    printf("%d", expectedSum-sum);
16    return 0;
17 }
```

Question 11.3

Complete the following C program that does the following operations one by one :

- copy str1 into str3, then print str3 ...use strcpy
- concatenate str1 and str2, and assign result to str3 and then print str3 ...use strcat
- print total length of str3 after concatenation ...use strlen function

char *strcpy(char *dest, const char *src)

char *strcat(char *dest, const char *src)

size_t strlen(const char *str);

For example :

Result
The copied str3 is: Hello
The concatenated str3 is: HelloWorld
The length of str3: 10

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main () {
5
6     char str1[20] = "Hello";
7     char str2[20] = "World";
8     char str3[]="";
9     int len;
10
11     // copy str1 into str3
12     strcpy(str3, str1);
13     printf("The copied str3 is: %s\n", str3);
14     // concatenate str1 and str2 and assign result to str3
15     // note that str3 has already contents of str1
16     strcat(str3, str2);
17     printf("The concatenated str3 is: %s\n", str3);
18     // print total length of str3 after concatenation
19     len = strlen(str3);
20     printf("The length of str3: %d\n", len);
21
22     return 0;
23 }
```

Question 11.4

Write a C program which reads a string from the user and then prints only N chars of a String starting from position M as a substring on the screen.

Read at first the string, then the numbers M and N respectively.

Hint: You do not need any built-in string function for this. Instead, you can read the input string char by char. Then, construct substring from that string. Do not forget to put Null Terminator (i.e. `'\0'`) to the end of your substring!

For example :

Input	Result
I love C++ Programming 4 9	ove C++ P

```
1 #include <stdio.h>
2
3 int main()
4 {
5     char string[1000], sub[1000];
6     int position, length, c = 0;
7
8     //read the string at first
9     fgets(string, sizeof string, stdin);
10
11    //get the position (M) and length (N) of substring\n");
12    scanf("%d%d", &position, &length);
13
14    while (c < length) {
15        sub[c] = string[position+c-1];
16        c++;
17    }
18
19    //set the last character as null terminator
20    sub[c] = '\0';
21
22    printf("%s\n", sub);
23
24    return 0;
25 }
```

Question 11.5

Write a C program which reads a string from the user and then reads a character from the user to find its frequency in the given string.
Read at first the string, then the character for finding its frequency.

For example :

Input	Result
I love C++ Programming o	Frequency of o = 2

```
1 #include <stdio.h>
2 int main()
3 {
4     char str[1000], ch;
5     int i, frequency = 0;
6     //read the string at first
7     fgets(str, sizeof str, stdin);
8
9     //read a character to find its frequency
10    scanf("%c",&ch);
11
12    //iterate over the string, i.e. read string char by char
13    //increment your counter , if you encounter the character
14    //given
15    for(i = 0; str[i] != '\0'; ++i)
16    {
17        if(ch == str[i])
18            ++frequency;
19    }
20
21    //print the frequency of desired character
22    printf("Frequency of %c = %d", ch, frequency);
23    return 0;
24 }
```

Question 11.6

Write a C program which reads a set of words from the user and then sorts these words in dictionary order. You can use the following string functions:
int strcmp(const char *str1, const char *str2) This function return values that are as follows,

- if Return value less than 0 then it indicates str1 is less than str2.
-
- if Return value greater than 0 then it indicates str2 is less than str1.
-
- if Return value = 0 then it indicates str1 is equal to str2.

char *strcpy(char *dest, const char *src)

For example :

Input	Result
C	C
C++	C++
Java	Java
PHP	JavaScript
Python	PHP
Perl	PHP
Ruby	Perl
R	Python
JavaScript	R
PHP	Ruby

```
1 #include <stdio.h>
2 #include <string.h>
3 int main()
4 {
5     int i, j;
6     char str[10][50], temp[50];
7
8     //read 10 words from the user & store each word in the
9     string array
10    for(i=0; i<10; ++i)
11        scanf("%s[^\n]", str[i]);
12
13    //Bubble sort
14    for(i=0; i<9; ++i)
15        for(j=i+1; j<10 ; ++j)
```

```

15     {
16         //compare strings
17         if(strcmp(str[i], str[j])>0)
18         {
19             //replace string at location i with j
20             strcpy(temp, str[i]);
21             strcpy(str[i], str[j]);
22             strcpy(str[j], temp);
23         }
24     }
25     //print sorted list of words
26     for(i=0; i<10; ++i)
27     {
28         puts(str[i]);
29     }
30     return 0;
31 }

```

Lab #12

In this lab session, students will explore the organization of tabular data through multidimensional arrays and the creation of custom data types using structures:

- **Multidimensional Array Management:** Mastering the declaration and initialization of 2-D arrays to represent data in grid or matrix formats.
- **Matrix Transformations:** Implementing algorithms to process matrix elements, including the calculation of a matrix transpose by swapping row and column indices.
- **Data Grouping with Structures:** Defining `struct` templates to aggregate variables of different data types into a single logical entity, such as student or book records.
- **Type Abstraction:** Utilizing the `typedef` keyword to create aliases for complex structures, enhancing code clarity and ease of maintenance.
- **Structural Operations in Functions:** Learning to pass structures to functions to perform specialized tasks, such as arithmetic operations on complex numbers.
- **Formatted Tabular Output:** Developing nested loop structures to display multidimensional data as clean, readable tables on the console.

Question 12.1

Complete the following C program that initializes of multidimensional array.

For example :

Result
0 1 2 3
4 5 6 7
8 9 10 11
a[0][0] = 0
...
a[4][0] = 4
a[4][1] = 8

```
1 #include <stdio.h>
2 int main(){
3
4     int arr[3][4] = {
5         {0, 1, 2, 3} ,    /* initializers for row indexed by
6     0 */
7         {4, 5, 6, 7} ,    /* initializers for row indexed by
8     1 */
9         {8, 9, 10, 11}    /* initializers for row indexed by
10    2 */
11    };
12
13    /*print arr, two dimensional array elements as a table
14    like in the following:
15    * 0 1 2 3
16    * 4 5 6 7
17    * 8 9 10 11
18    */
19    int i, j;
20    for ( i = 0; i < 3; i++ ) {
21        for ( j = 0; j < 4; j++ ) {
22            printf("%d ", arr[i][j] );
23        }
24        printf("\n");
25    }
26
27    int a[5][2] = { {0,0}, {1,2}, {2,4}, {3,6},{4,8}};
28
29    /* print a, array elements with indexes as in the
30    following:
```

```
26     * a[0][0] = 0
27     * a[0][1] = 0
28     * a[1][0] = 1
29     * .....
30     */
31     for ( i = 0; i < 5; i++ ) {
32         for ( j = 0; j < 2; j++ ) {
33             printf("a[%d][%d] = %d\n", i,j, a[i][j] );
34         }
35     }
36 }
```

Question 12.2

Complete the following C program that reads 2*3 2-D array elements from the user.

Row and Column sizes are fixed to 2 and 3 respectively.

For example :

Input	Result
2 5 6 12 -23 45	Two Dimensional array elements: 2 5 6 12 -23 45

```
1 #include<stdio.h>
2 int main(){
3     /* 2D array declaration*/
4     int disp[2][3];
5     /*Counter variables for the loop*/
6     int i, j;
7     /*read 2*3=6 array elements from the user one by one*/
8     for(i=0; i<2; i++) {
9         for(j=0; j<3; j++) {
10             scanf("%d", &disp[i][j]);
11         }
12     }
13     /*Displaying array elements, row by row*/
14     printf("Two Dimensional array elements:\n");
15     for(i=0; i<2; i++) {
16         for(j=0; j<3; j++) {
17             printf("%d ", disp[i][j]);
18         }
19         printf("\n");
20     }
21     return 0;
22 }
```

Question 12.3

Complete the following C program that reads 2-D array elements from the user and then calculates the transpose of the given matrix.

Read row and column values from the user at first. Then read the array elements one by one.

USE THE GIVEN TEMPLATE! DO NOT DISRUPT IT!

Hint: The transpose of a matrix is obtained by changing its rows into columns (or equivalently, its columns into rows). A rectangular array of numbers or functions that are arranged in the form of rows and columns is called a matrix. This array of numbers are called either entries or elements of a matrix.

$$A = \begin{bmatrix} a & b & c \\ d & e & f \end{bmatrix}_{2 \times 3} \quad A^T = \begin{bmatrix} a & d \\ b & e \\ c & f \end{bmatrix}_{3 \times 2}$$

For example :

Input	Result
2 3 2 5 6 12 -23 45	2 5 6 12 -23 45 2 12 5 -23 6 45

```
1 #include <stdio.h>
2
3 int main()
4 {
5
6     /*Declare two 2-D arrays with the size of 10x10*/
7     int a[10][10], transpose[10][10], r, c, i, j;
8
9     /*read number of rows and columns*/
10    scanf("%d %d", &r, &c);
11
12    /*read r*c array elements from the user one by one*/
13    for(i=0; i<r; ++i)
14        for(j=0; j<c; ++j)
15            scanf("%d", &a[i][j]);
16
17    /*Display the r*c matrix */
18    for(i=0; i<r; ++i){
19        for(j=0; j<c; ++j)
```

```

20     {
21         printf("%d ", a[i][j]);
22     }
23     printf("\n");
24 }
25
26 /*Calculate the transpose of r*c matrix and fill transpose
27 2-D array*/
28 for(i=0; i<r; ++i)
29     for(j=0; j<c; ++j)
30         transpose[j][i] = a[i][j];
31
32 /*Display the transpose of r*c matrix */
33 for(i=0; i<c; ++i){
34     for(j=0; j<r; ++j)
35         printf("%d ",transpose[i][j]);
36     printf("\n");
37 }
38 return 0;
39 }

```

Question 12.4

Complete the following C program that creates student structure then assign an ID number to that structure. After all, print that student IDs. Set first student's ID as 1020 and name as "Ali", set second student's ID as 1021 and name as "Veli".

For example :

Result

Student-1 name: Ali and ID: 1020

Student-2 name: Veli and ID: 1021

```
1 #include <stdio.h>
2 #include <string.h>
3 struct Student{
4     int id; //Student ID as an integer
5     char name[50]; //Char array, i.e. string as the student
        name
6 };
7 int main(){
8     struct Student Stu1; //create student1
9     struct Student Stu2; //create student2
10
11     Stu1.id=1020; //set student-1's ID as 1020
12     Stu2.id=1021; //set student-2's ID as 1021
13
14     strcpy(Stu1.name, "Ali"); //set student-1's name as Ali
15     strcpy(Stu2.name, "Veli"); //set student-2's name as Veli
16
17     //print information about student-1
18     printf("Student-1 name: %s and ID: %d\n",Stu1.name, Stu1.id
        );
19
20     //print information about student-2
21     printf("Student-2 name: %s and ID: %d",Stu2.name, Stu2.id);
22 }
```

Question 12.5

Write a C program that has Book Structure with id number. Inside the main create a 2 book and assign an id number with as 1213 and 1214. (Use typedef and newline while printing)

For example :

Result

1213

1214

```
1 #include <stdio.h>
2 #include <string.h>
3 typedef struct Books{
4     int id;
5     char title[50];
6 }Kitap;
7 int main(){
8     Kitap Book1;
9     Kitap Book2;
10    Book1.id=1213;
11    Book2.id=1214;
12    printf("%d",Book1.id);
13    printf("\n%d",Book2.id);
14
15 }
```

Question 12.6

Complete the following C program that creates a structure for complex numbers that has real and imaginary float variable. Then compose a add function to add these two complex number to print. Get complex numbers from the user one by one.

For example :

Input	Result
4.1 5.1 3.0 4.1	Sum = 7.1 + 9.2i

```
1  #include <stdio.h>
2  typedef struct complex
3  {
4      float real;
5      float imag;
6  } complex;
7  complex add(complex n1,complex n2);
8
9  int main()
10 {
11     complex n1, n2, temp;
12     scanf("%f %f", &n1.real, &n1.imag);
13     scanf("%f %f", &n2.real, &n2.imag);
14     temp=add(n1, n2);
15     printf("Sum = %.1f + %.1fi", temp.real, temp.imag);
16
17     return 0;
18 }
19 complex add(complex n1, complex n2)
20 {
21     complex temp;
22     temp.real = n1.real + n2.real;
23     temp.imag = n1.imag + n2.imag;
24     return(temp);
25 }
```


Lab #13

In this lab session, students will master the concepts of pointers, memory addressing, and their critical role in data manipulation:

- **Pointer Mechanics:** Understanding how to declare pointer variables and utilize the address-of (&) and indirection (*) operators to access memory.
- **Memory Complementarity:** Exploring the relationship between operators to prove that & and * are inverse operations.
- **Pointer-Array Duality:** Accessing array elements using pointer arithmetic (*(base + offset)) as a functional alternative to standard indexing.
- **Indirect Input and Aggregation:** Performing input operations and data processing, such as calculating the sum of array elements, strictly through pointer-based references.
- **Passing by Reference:** Implementing functions that modify the original data of the caller by receiving memory addresses as arguments.
- **Double Pointers:** Introduction to the syntax and utility of pointers to pointers (**ptr) for multi-level memory indirection.
- **Reference-Based Data Swapping:** Developing modular functions to exchange values between variables by manipulating their underlying memory locations.

Question 13.1

In this question declare an integer an assign value of 5 and print that integer by pointer.

```
1 #include <stdio.h>
2 int main(){
3     int x=5;
4     int *xPtr;
5     xPtr=&x;
6     printf("%d",*xPtr);
7 }
```

Question 13.2

& and * are complements of each other. Prove that & and * are complements of each other by initializing and printing value of 5.

For example :

Result
5

```
1 #include <stdio.h>
2 int main(){
3     int data[5];
4
5     scanf("%d",data+1);
6     printf("%d",*(data+1));
7 }
```

Question 13.3

Write A C program that takes an integer input from user to assign second element of array then print that value to the screen.

For example :

Input	Result
9	9
0	0

```
1 #include <stdio.h>
2 int main(){
3     int data[5];
4
5     scanf("%d",data+1);
6     printf("%d",*(data+1));
7 }
```

Question 13.4

Write a complete C program that takes inputs of array with size of 6. Then print the summation of all array elements. Do input operations by pointers.

For example :

Input	Result
0 1 2 3 4 5	15

```
1 #include <stdio.h>
2 int main()
3 {
4     int i, classes[6], sum = 0;
5     for(i = 0; i < 6; ++i)
6     {
7         // (classes + i) is equivalent to &classes[i]
8         scanf("%d", (classes + i));
9         // *(classes + i) is equivalent to classes[i]
10        sum += *(classes + i);
11    }
12    printf("%d", sum);
13    return 0;
14 }
```

Question 13.5

Get an integer value from the user and print the incremented value of that integer by using address of pointer function.

For example :

Input	Result
0	1

```
1 #include <stdio.h>
2 void f(int **in) {
3     (**in)++;
4 }
5 int main() {
6     int i;
7     scanf("%d",&i);
8     int *iPtr = &i;
9     int **jPtr= &iPtr;
10    f(jPtr);
11    printf("%d", i);
12    return 0;
13 }
```

Question 13.6

Get an integer value from the user and print incremented value of that integer by using function operation.

For example :

Input	Result
20	21
0	1
-21	-20

```
1 #include <stdio.h>
2 void f(int *j) {
3     (*j)++;
4 }
5 int main() {
6     int i;
7     scanf("%d",&i);
8     int *p = &i;
9     f(p);
10    printf("%d", i);
11    return 0;
12 }
```

Question 13.7

Write a complete C program that initializes two integer numbers and swaps them by using pointer function.

For example :

Result
10
5

```
1 #include <stdio.h>
2 void swap(int *n1, int *n2);
3 int main()
4 {
5     int num1 = 5, num2 = 10;
6     swap( &num1, &num2);
7     printf("%d\n", num1);
8     printf("%d", num2);
9     return 0;
10 }
11 void swap(int * n1, int * n2)
12 {
13     int temp;
14     temp = *n1;
15     *n1 = *n2;
16     *n2 = temp;
17 }
```


Lab #14

In this final lab session, students will explore the advanced integration of structures with pointers and the essential techniques of dynamic memory management:

- **Pointer-to-Structure Interaction:** Mastering the syntax for accessing structure members via pointers using the arrow operator (`->`) and dereferencing techniques.
- **Efficient Data Passing:** Implementing functions that accept structure pointers to modify original data and avoid the overhead of copying large data structures.
- **Dynamic Memory Allocation (`malloc`):** Learning to request specific blocks of memory at runtime from the heap to handle variable-sized data sets.
- **Contiguous Allocation (`calloc`):** Utilizing `calloc` for memory allocation with automatic initialization to zero, ensuring predictable default states.
- **Advanced Structural Logic:** Developing complex algorithms to handle real-world units, such as calculating time differences and distance summations using pointer-based structure functions.
- **Runtime Data Structures:** Creating and managing arrays of structures dynamically based on user-defined counts, emphasizing flexible memory use.
- **Memory Safety and Casting:** Understanding the importance of proper type casting when allocating memory and managing pointer references within the heap.

Question 14.1

Write a complete C program that has structure named with person. Person structure has age(int) and weight(float) parameter. Get and print the age and weight values from user by using pointer operation.

For example :

Input	Result
32 74.5	32 74.500000

```
1 #include <stdio.h>
2 struct person
3 {
4     int age;
5     float weight;
6 };
7 int main()
8 {
9     struct person *personPtr, person1;
10    personPtr = &person1; // Referencing pointer to memory
                             address of person1
11    scanf("%d%f", &(*personPtr).age, &(*personPtr).weight);
12    printf("%d %f", (*personPtr).age, (*personPtr).weight);
13    return 0;
14 }
```

Question 14.2

Write a complete C program that has distance structure. Structure has feet and inch parameter. Then calculate the summation of 2 distance that should be taken from user.

- first input is the first number's distance feet,
- second input first number's distance inch,
- third input second number's distance feet,
- fourth input second number's distance inch

Note that 1 foot is equal to 12 inches: 1ft = 12" or 1' = 12"

For example :

Input	Result
2 5 5 7	Sum of distances = 8'-0.0"

```
1 #include <stdio.h>
2 struct distance
3 {
4     int feet;
5     float inch;
6 };
7 void add(struct distance d1, struct distance d2, struct
8         distance *d3);
9 int main()
10 {
11     struct distance dist1, dist2, dist3;
12     scanf("%d", &dist1.feet);
13     scanf("%f", &dist1.inch);
14
15     scanf("%d", &dist2.feet);
16     scanf("%f", &dist2.inch);
17
18     add(dist1, dist2, &dist3);
19
20     //passing structure variables dist1 and dist2 by value
21     //whereas passing structure variable dist3 by reference
22     printf("Sum of distances = %d\'-%.1f\"", dist3.feet,
23           dist3.inch);
24
25     return 0;
26 }
27 void add(struct distance d1, struct distance d2, struct
28         distance *d3)
29 {
```

```
26     //Adding distances d1 and d2 and storing it in d3
27     d3->feet = d1.feet + d2.feet;
28     d3->inch = d1.inch + d2.inch;
29
30     if (d3->inch >= 12) {          /* if inch is greater or equal
31                                     to 12, converting it to feet. */
32         d3->inch -= 12;
33         ++d3->feet;
34     }
```

Question 14.3

Write a complete C program that has TIME structure. That structure has hour minute and second. Get hours, minutes and seconds respectively from user and calculate the time difference by pointer function.

For example :

Input	Result
5 8 7 2 5 4	3:3:3

```
1 #include <stdio.h>
2 struct TIME
3 {
4     int seconds;
5     int minutes;
6     int hours;
7 };
8 void differenceBetweenTimePeriod(struct TIME t1, struct TIME
    t2, struct TIME *diff);
9
10 int main()
11 {
12     struct TIME startTime, stopTime, diff;
13     scanf("%d %d %d", &startTime.hours, &startTime.minutes, &
    startTime.seconds);
14     scanf("%d %d %d", &stopTime.hours, &stopTime.minutes, &
    stopTime.seconds);
15
16     // Calculate the difference between the start and stop
    time period.
17     differenceBetweenTimePeriod(startTime, stopTime, &diff);
18     printf("%d:%d:%d", diff.hours, diff.minutes, diff.seconds
    );
19
20     return 0;
21 }
22
23 void differenceBetweenTimePeriod(struct TIME start, struct
    TIME stop, struct TIME *diff)
24 {
25     if(stop.seconds > start.seconds){
26         --start.minutes;
27         start.seconds += 60;
28     }
29 }
```

```
30 diff->seconds = start.seconds - stop.seconds;
31 if(stop.minutes > start.minutes){
32     --start.hours;
33     start.minutes += 60;
34 }
35
36 diff->minutes = start.minutes - stop.minutes;
37 diff->hours = start.hours - stop.hours;
38 }
```

Question 14.4

Write a C program that allocates memory for five integers. After that, initialize int to the value of 1585323. Print that value on the screen.

For example :

Result

1585323

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main()
5 {
6     int i=1585323;
7     int *Aptr;
8     Aptr=malloc(sizeof(int)*5);
9     Aptr=&i;
10    printf("%d",*Aptr);
11    return 0;
12 }
```

Question 14.5

Write a C program that allocate 3 times size of integer. Then verify that calloc declare variables '0' as default.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main() {
5     int *p;
6     p = calloc(3, sizeof(int));
7     printf("%d", p[1]);
8 }
```


Question 14.6

Write a complete C program that has person structure. Structure has int age and weight. Inside the main program asks for person number to allocate memory accordingly. Get the age and weight values from the user and print them to the screen.

For example :

Input	Result
2 15 48 16 50	15 48
	16 50

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct person {
5      int age;
6      int weight;
7  };
8
9  int main()
10 {
11
12     struct person *ptr;
13     int i, num;
14
15     scanf("%d", &num);
16     ptr = (struct person*) malloc(num * sizeof(struct person)
17 );
18
19     for (i = 0; i < num; ++i){
20         scanf("%d%d",&(ptr+i)->age,&(ptr+i)->weight);
21     }
22
23     for(i=0;i<num;++i){
24         printf("%d %d\n",(ptr+i)->age, (ptr+i)->weight);
25     }
26     return 0;
27 }
```

REFERENCES

- [1] Kernighan & Ritchie, The C Programming Language, Prentice Hall
Deitel & Deitel, C How to Program, Prentice Hall